Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Che cosa seleziona `.art p:nth-child(2)` nel frammento seguente?

```
<div class="art">
  <h2>Titolo</h2>
  <p>Testo 2</p>
  <h2>Titolo</h2>
  <p>Testo 4</p>
  <p>Testo 5</p>
  <p>Testo 6</p>
  <p>Testo 7</p>
</div>
```

- [x] A — Il 2° figlio del `div`, che è un paragrafo.
- [ ] B — Tutti i paragrafi del `div` dal 2° in poi.
- [ ] C — Nessun elemento: `:nth-child` non si può combinare con un tipo.
- [ ] D — Il 2° figlio di ogni paragrafo.

Risposta: A. `:nth-child(k)` conta **tutti** i figli del genitore, di qualunque tipo; poi il selettore di tipo `p` controlla che quello in posizione k sia un paragrafo. Qui il 2° figlio è un `p`. *(Dispensa: cap. 3, «Il conteggio senza selettore di tipo»)*

Es. **2.** **(2 punti)** Quale dei seguenti modi di includere uno script esterno è corretto?

- [ ] A — `<script src="app.js" />`
- [ ] B — `<link rel="script" href="app.js">`
- [ ] C — `<script href="app.js"></script>`
- [ ] D — `<script>app.js</script>`
- [ ] E — `<js src="app.js"></js>`
- [x] F — `<script src="app.js"></script>`

Risposta: F. Il tag `script` non è mai auto-chiudente: anche con `src` va chiuso con `</script>`. `<script src="app.js" />` viene letto come un tag aperto, e il resto della pagina finisce dentro lo script. *(Dispensa: cap. 5, «Includere JavaScript in una pagina»)*

Es. **3.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul pulsante «Uno» e poi su «Due»?

```
<button id="b1">Uno</button>
<button id="b2">Due</button>
<script>
  document.getElementById("b1").addEventListener("click", function () {
    console.log(this.id);
  });
  document.getElementById("b2").addEventListener("click", () => {
    console.log(this === window);
  });
</script>
```

- [ ] A — `b1`, `undefined`.
- [ ] B — `window`, `true`.
- [ ] C — `b1`, `false`.
- [ ] D — `undefined`, `false`.
- [ ] E — `undefined`, `true`.
- [x] F — `b1`, `true`.

Risposta: F. In un gestore definito con `function`, `this` è l'elemento a cui il gestore è stato assegnato. Le *arrow function* non hanno un `this` proprio e usano quello del codice in cui sono scritte: qui il livello globale dello script, dove `this` è `window`. *(Dispensa: cap. 7, «Il valore di this in un gestore»)*

SOLUZIONI

Es. **4.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log("1");
2 const p = new Promise((resolve) => {
3   console.log("2");
4   resolve("3");
5   console.log("4");
6 });
7 p.then((v) => console.log(v));
8 console.log("5");
```

- A 1, 2, 3, 4, 5.
- B 1, 2, 5, 3.
- C 1, 2, 4, 3, 5.
- D 1, 5, 2, 4, 3.
- **E** 1, 2, 4, 5, 3.
- F 1, 5, 2, 3, 4.

Risposta: E. L'executor viene invocato **subito**, dal costruttore: 2 e 4 sono stampati durante la creazione della Promise. `resolve` sistema la Promise ma non interrompe l'executor. Il gestore registrato con `then` viene eseguito solo dopo la fine del codice sincrono, anche se la Promise è già risolta. *(Dispensa: cap. 8, «Creare una Promise»)*

Es. **5.** **(2 punti)** Quale codice di stato dovrebbe restituire il server nella seguente situazione: un utente **autenticato**, ma senza i permessi necessari, chiede una pagina riservata agli amministratori?

- A `503 Service Unavailable`
- **B** `403 Forbidden`
- C `405 Method Not Allowed`
- D `404 Not Found`
- E `401 Unauthorized`
- F `400 Bad Request`

Risposta: B. 401 vuol dire «non so chi sei» (manca l'autenticazione), 403 «so chi sei, ma non puoi». 301 e 302 sono spostamenti definitivo e temporaneo; 304 dice che la copia in cache è ancora buona. 502 segnala una risposta non valida ricevuta da un server intermedio, 503 un server momentaneamente non disponibile. 201 accompagna la creazione di una risorsa, 204 un successo senza corpo, 405 un metodo non ammesso su un percorso esistente. *(Dispensa: cap. 9, «La risposta»)*

Es. **6.** **(2 punti)** Si consideri il template Pug `pagina.pug` riportato di seguito, trasformato in HTML con la chiamata a destra. Quale codice HTML viene generato?

```
1 if !user
2   a(href="/login") Accedi
3 else if user.isAdmin
4   p Ai tuoi ordini, #{user.name}
5 else
6   p Bentornato, #{user.name}
```

```
1 pug.renderFile("pagina.pug", {
2   user: { name: "Ada", isAdmin: false }
3 });
```

- A `<a href="/login">Accedi</a>`
- B `<p>Ai tuoi ordini, Ada</p>`
- C `<p>Bentornato, user.name</p>`
- D `<a href="/login">Accedi</a><p>Ai tuoi ordini, Ada</p><p>Bentornato, Ada</p>`
- **E** `<p>Bentornato, Ada</p>`
- F `<p>Ai tuoi ordini, Ada</p><p>Bentornato, Ada</p>`

Risposta: E. In un condizionale Pug finisce nell'HTML **solo** il ramo scelto. Se `user` manca, `!user` è vero e si mostra il link di accesso (le espressioni degli altri rami non vengono nemmeno valutate); altrimenti si sceglie in base a `user.isAdmin`. *(Dispensa: cap. 12, «Condizionali»)*

SOLUZIONI

Es. **7.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function parseCookies(header) {
  const list = {};
  header.split(";").forEach((cookie) => {
    let [name, ...rest] = cookie.split("=");
    name = name.trim();
    const value = rest.join("=").trim();
    if (!name || !value) return;
    list[name] = decodeURIComponent(value);
  });
  return list;
}
const c = parseCookies("auth=true; username=luigi; tema=a=b; vuoto=");
console.log(Object.keys(c).length, c.username, c.tema, typeof c.auth);
```

- A `4 luigi a=b string.`
- B `3 luigi b string.`
- C `3 luigi a=b boolean.`
- D `3 luigi a string.`
- **E** `3 luigi a=b string.`
- F `4 luigi a boolean.`

Risposta: E. L'header `Cookie` contiene coppie `nome=valore` separate da `;`. Il cookie `vuoto` non ha valore e viene scartato. Ricomponendo con `join("=")` tutto ciò che segue il primo `=`, il valore di `tema` resta `a=b`. I valori dei cookie sono stringhe: `auth` vale `"true"`, non il booleano `true`. *(Dispensa: cap. 13, «Recuperare le informazioni di sessione»)*

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /a`, `GET /b`, `GET /c` e `GET /d`?

```
import express from "express";

const app = express();

app.get("/a", (req, res) => res.send({ ok: true }));
app.get("/b", (req, res) => res.sendStatus(404));
app.get("/c", (req, res) => res.status(201).send("creato"));
app.get("/d", (req, res) => res.redirect("/a"));

app.listen(3000);
```

- A `200 [object Object], 404 Not Found, 201 creato, 302 -> /a.`
- B `200 {"ok":true}, 404 , 201 creato, 302 -> /a.`
- C `200 {"ok":true}, 404 Not Found, 201 creato, 200 {"ok":true}.`
- **D** `200 {"ok":true}, 404 Not Found, 201 creato, 302 -> /a.`
- E `200 {"ok":true}, 200 404, 201 creato, 301 -> /a.`
- F `200 {"ok":true}, 404 Not Found, 200 creato, 302 -> /a.`

Risposta: D. `res.send` con un oggetto lo converte in JSON (e imposta `Content-Type: application/json`); `res.sendStatus` imposta il codice e invia come corpo la sua rappresentazione testuale; `res.status` imposta il codice e restituisce `res`, per concatenare l'invio; `res.redirect` risponde con un 302 e l'header `Location` (il browser seguirebbe da solo il reindirizzamento). *(Dispensa: cap. 15, «Rispondere al client»)*

Es. **9.** **(5 punti)** Perché sono stati introdotti i **moduli** in JavaScript? Si descrivano le forme di `export` e `import` e il comportamento degli script di tipo modulo nel browser.

Traccia della risposta.

- Senza moduli tutti gli script condividono lo scope globale: conflitti di nomi e dipendenze implicite fra file.
- Un modulo ha un proprio scope ed esporta esplicitamente ciò che vuole condividere: export con nome (`export const x`) ed export predefinito (`export default`).
- `import { x, y as z } from './m.js'` per gli export con nome, `import M from './m.js'` per quello predefinito.
- Nel browser: `<script type="module">`, strict mode, esecuzione differita, una sola valutazione per modulo; servono un server (no `file://`).

(Dispensa: cap. 6, «Moduli»)

SOLUZIONI

Es. **10.** **(5 punti)** Quali sono gli svantaggi (se ve ne sono) dell'utilizzo di un approccio *multi-threaded request handling* per un'applicazione web che effettua frequenti accessi in lettura/scrittura a un database? Motivare la risposta.

Traccia della risposta.

- Ogni richiesta occupa un thread (o processo) del pool per tutta la sua durata, anche mentre attende le operazioni di I/O sul database.
- Con accessi frequenti i thread passano la maggior parte del tempo bloccati: inefficienza (memoria e risorse spese per aspettare).
- Scalabilità limitata: il pool ha dimensione fissa, e non si servono più richieste concorrenti di quanti sono i thread (il ristorante con un cameriere per tavolo).
- Alternativa: modello a eventi con I/O non bloccante (Node.js): un thread non aspetta mai, l'I/O procede altrove e la sua callback torna all'event loop.

(Dispensa: cap. 11, «Il modello multi-thread») — Fonte: prima prova intercorso del 23 aprile 2024.

Es. **11.** **(5 punti)** Che cosa sono le *custom property* del CSS? Se ne descrivano dichiarazione e uso, il rapporto con cascata ed ereditarietà, i valori di riserva e la regola `@property`.

Traccia della risposta.

- Valori definiti da chi scrive il foglio di stile e riutilizzabili: nome con due trattini (`--primary-color: #663399`, maiuscole significative), letti con `var()`.
- Partecipano alla cascata, sono ereditate e si possono sovrascrivere: definite in `:root` valgono per tutta la pagina, ridefinite in un elemento valgono per i suoi discendenti (temi, modalità scura con una media query).
- `var(--nome, riserva)`: il secondo argomento si usa se la proprietà non è definita.
- `@property` registra la proprietà con tipo (`syntax`), ereditarietà (`inherits`) e valore iniziale (`initial-value`).

(Dispensa: cap. 14, «Custom properties»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	A	3	pseudo-classi e :nth-child
2	F	5	includere JavaScript
3	F	7	eventi
4	E	8	Promise
5	B	9	codici di stato
6	E	12	template Pug
7	E	13	lo stato nei cookie
8	D	15	rispondere al client
9	aperta	6	moduli
10	aperta	11	il modello multi-thread
11	aperta	14	custom property

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI